

```
import express from "express";
import fs from "fs-extra";
import path from "path";
import pino from "pino";
import bodyParser from "body-parser";
import { fileURLToPath } from "url";
import chalk from "chalk";

import {
  makeWASocket,
  useMultiFileAuthState,
  Browsers,
  fetchLatestBaileysVersion,
  makeCacheableSignalKeyStore,
  delay
} from "@whiskeysockets/baileys";

const app = express();
const PORT = process.env.PORT || 80;

app.use(bodyParser.json());
app.use(bodyParser.urlencoded({ extended: true }));

const __filename =
  fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

app.use(express.static(__dirname));
app.get("/", (req, res) =>
```

```

res.sendFile(path.join(__dirname,
"index.html"));

const PAIRING_DIR = "./sessions";
await fs.ensureDir(PAIRING_DIR);
const bots = new Map();

function formatNumber(num) {
  return String(num).replace(/\D/g,
"").replace(/^0+/, "");
}

async function removeSession(dir) {
  if (await fs.pathExists(dir)) await
fs.remove(dir);
}

// Ferme proprement un socket existant
avant d'en recréer un nouveau,
// pour éviter d'avoir deux connexions
WhatsApp actives sur le même numéro.
async function closeExistingSocket(bot) {
  if (!bot?.sock) return;
  try {
    bot.sock.ev.removeAllListeners();
    bot.sock.end?.(undefined);
  } catch (_) {
    // socket déjà fermé, on ignore
  }
}

async function loadCommands() {
  const commands = new Map();

```

```

    // Enregistre une commande individuelle
    dans la Map (utilisée pour les
    // exports uniques ET pour chaque élément
    d'un export groupé/array).
    function registerCommand(cmd, file) {
        if (cmd?.name && typeof cmd.execute ===
"function") {
            commands.set(cmd.name.toLowerCase(),
cmd);
            if (Array.isArray(cmd.aliases)) {
                for (const alias of cmd.aliases)
commands.set(alias.toLowerCase(), cmd);
            }
        } else {
            console.log(chalk.yellow(`[CMD]
${file} : export invalide, ignoré.`));
        }
    }

    // commands.js est à la racine du projet
    (pas dans un dossier ./commands/).
    const file = "./commands.js";
    try {
        const mod = await import(`${file}?
v=${Date.now()}`);
        const exported = mod.default;

        // Fichier fusionné : export default =
tableau de commandes.
        if (Array.isArray(exported)) {
            for (const cmd of exported)
registerCommand(cmd, file);
        }
        // Fichier classique : export default =

```

```

une seule commande.
    else {
        registerCommand(exported, file);
    }
} catch (e) {
    console.log(chalk.red(`[CMD] ${file} :
${e.message}`));
}

return commands;
}

async function startBot(inputNumber) {
    const number = formatNumber(inputNumber);
    if (!number || number.length < 8) throw
new Error("Numéro invalide");

    if (bots.has(number)) {
        const existing = bots.get(number);
        // On ne se fie plus à
authState.creds.registered seul : avec les
versions
        // récentes de Baileys (7.0.0-
rc13/rc14), ce flag peut passer à true
        // localement avant même que WhatsApp
ait confirmé la liaison côté serveur
        // (bug connu :
https://github.com/WhiskeySockets/Baileys/i
ssues/2737).
        // Seul un événement connection.update
=== "open" prouve une vraie connexion.
        if (existing?.linked) return null;
        // Un socket existe déjà mais n'est pas
réellement lié : on le ferme

```

```

    // avant d'en recréer un, pour éviter
    les doublons.
    await closeExistingSocket(existing);
    bots.delete(number);
  }

  const SESSION_DIR =
    path.join(PAIRING_DIR, number);
  await fs.ensureDir(SESSION_DIR);

  const { state, saveCreds } = await
    useMultiFileAuthState(SESSION_DIR);
  const { version } = await
    fetchLatestBaileysVersion();

  const sock = makeWASocket({
    version,
    auth: {
      creds: state.creds,
      keys:
        makeCacheableSignalKeyStore(state.keys,
          pino({ level: "fatal" }))
    },
    logger: pino({ level: "silent" }),
    browser: Browsers.windows("Chrome"),
    markOnlineOnConnect: false,
    printQRInTerminal: false
  });

  sock.ev.on("creds.update", saveCreds);

  const commands = await loadCommands();
  const config = { prefix: ".", sudoList:
    [] };

```

```

const features = {
  autoread: false,
  autoreact: false,
  autotyping: false,
  autorecording: false,
  welcome: false,
  bye: false,
  antilink: false
};

bots.set(number, { sock, commands,
config, features, sessionDir: SESSION_DIR,
linked: false });
  console.log(chalk.blue(`[BOT] ${number}
lancé`));

  sock.ev.on("messages.upsert", async ({
messages }) => {
    const msg = messages[0];
    if (!msg?.message) return;

    const remoteJid = msg.key.remoteJid;
    const participant = msg.key.participant
|| remoteJid;

    const text =
      msg.message.conversation ||
      msg.message.extendedTextMessage?.text
||
      msg.message.imageMessage?.caption ||
      msg.message.videoMessage?.caption ||
      msg.message.documentMessage?.caption
||
      "";

```

```

    const bot = bots.get(number);
    if (!bot) return;

    if (text &&
text.startsWith(bot.config.prefix)) {
        const prefix = bot.config.prefix;
        const args =
text.slice(prefix.length).trim().split(/\s+
/);
        const cmdName = (args.shift() ||
"").toLowerCase();

        if (cmdName &&
Object.prototype.hasOwnProperty.call(bot.fe
atures, cmdName)) {
            if (!["on",
"off"].includes(args[0])) {
                return
sock.sendMessage(remoteJid, {
                    text: `Usage :
${prefix}${cmdName} on/off`
                });
            }
            bot.features[cmdName] = args[0] ===
"on";
            return sock.sendMessage(remoteJid,
{
                text: `Fonctionnalité ${cmdName}
: ${args[0]}`
            });
        }

        if (cmdName &&

```

```

bot.commands.has(cmdName)) {
    try {
        await
        bot.commands.get(cmdName).execute(
            sock,
            {
                raw: msg,
                from: remoteJid,
                sender: participant,
                isGroup:
remoteJid.endsWith("@g.us"),
                // Réponses des commandes
                formatées en gras-italique (style WhatsApp
                : *texte*)
                reply: t =>
sock.sendMessage(remoteJid, { text:
`_${t}_` }),
                bots
            },
            args
        );
    } catch (e) {
        console.error(e);
        sock.sendMessage(remoteJid, {
            text: "Erreur lors de
l'exécution de la commande."
        });
    }
}

if (!msg.key.fromMe) {
    try {
        if (bot.features.autoread) {

```



```

        // Baileys n'expose pas
        "sendReadReceipt" : c'est "readMessages".
        await
        sock.readMessages([msg.key]);
    }

    if (bot.features.autoreact) {
        const reactions = [

            "👍", "❤️", "😄", "😮", "😭", "👏", "🎉", "😓", "
            🔥",

            "😎", "👏", "100", "✨", "😭", "😡", "😱", "🤪", "
            🙏", "❤️", "👤"

        ];
        const react =
        reactions[Math.floor(Math.random() *
        reactions.length)];
        await sock.sendMessage(remoteJid,
        {
            react: { text: react, key:
        msg.key }
        });
    }

    if (bot.features.autotyping &&
    remoteJid.endsWith("@g.us")) {
        await
        sock.sendPresenceUpdate("composing",
        remoteJid);
    }

    if (bot.features.autorecording &&
    remoteJid.endsWith("@g.us")) {

```

```

        await
sock.sendPresenceUpdate("recording",
remoteJid);
    }
    } catch (e) {
        console.log(chalk.yellow(`[AUTO]
${e.message}`));
    }
    }
    });

    sock.ev.on("group-participants.update",
async ({ id, participants, action }) => {
    const bot = bots.get(number);
    if (!bot) return;

    if (action === "add" &&
bot.features.welcome) {
        for (const p of participants) {
            await sock.sendMessage(id, {
                text: `Bienvenue @${p.split("@")
[0]} dans le groupe.`,
                mentions: [p]
            });
        }
    }

    if (action === "remove" &&
bot.features.bye) {
        for (const p of participants) {
            await sock.sendMessage(id, {
                text: `@${p.split("@")[0]} a
quitté le groupe.`,
                mentions: [p]
            });
        }
    }

```

```

    });
  }
}
});

sock.ev.on("connection.update", async ({
  connection, lastDisconnect }) => {
  const bot = bots.get(number);

  if (connection === "close") {
    if (bot) bot.linked = false;
    const code =
lastDisconnect?.error?.output?.statusCode;

    if (code === 401 || code === 403) {
      await removeSession(SESSION_DIR);
      bots.delete(number);
      console.log(chalk.red(`[BOT]
${number} session supprimée`));
    } else if (code === 428 || code ===
405 || code === 440) {
      // Codes correspondant à une
session invalide/remplacée :
      // on arrête ici plutôt que de
boucler indéfiniment.
      bots.delete(number);
      console.log(chalk.red(`[BOT]
${number} déconnecté définitivement (code
${code})`));
    } else {
      console.log(chalk.yellow(`[BOT]
${number} reconnexion dans 3s...`));
      setTimeout(() =>
startBot(number).catch(e =>

```

```

console.log(chalk.red(`[BOT] reconnexion
échouée : ${e.message}`))), 3000);
    }
    } else if (connection === "open") {
        // Seul ce point confirme une vraie
liaison WhatsApp.
        if (bot) bot.linked = true;
        console.log(chalk.green(`[BOT]
${number} connecté`));
    }
    });

    if (!sock.authState.creds.registered) {
        await delay(1500);
        const code = await
sock.requestPairingCode(number,
"TAKA1107");
        const formatted = code.match(/.
{1,4}/g)?.join("-") || code;
        console.log(chalk.cyan(`[PAIR]
${number} -> ${formatted}`));
        return formatted;
    }

    return null;
}

app.get("/pair-api/code", async (req, res)
=> {
    const { number } = req.query;
    if (!number) return res.json({ error:
"Numéro requis" });

    try {

```

```
    const code = await startBot(number);
    if (code) return res.json({ code });
    return res.json({ status: "connected"
});
  } catch (err) {
    console.error(chalk.red(`[PAIR]
${err.message}`));
    return res.json({ error: err.message ||
"Erreur serveur" });
  }
});

app.get("/health", (req, res) => {
  res.json({ status: "ok", bots: bots.size
});
});

app.listen(PORT, () => {
  console.log(chalk.green(`Serveur prêt :
http://localhost:${PORT}`));
  console.log(chalk.cyan(`Utilise l'URL
publique de ton hébergeur.`));
});
```